F-ComFlow

Developer Implementation Guide — Phased Build Plan

Audience: Development team (backend, frontend, AI/ML)
Target: Working live demo for SMUCT CSE FEST 2026 — Software Project Showcase
Document version: 1.0 • July 3, 2026

How to Use This Guide

The build is divided into **8 phases (Phase 0–7)**. Phases must be completed **in order**: each one produces a stable, testable slice of the product that the next phase builds on. At the end of every phase there is an **Exit Gate** — a checklist of things that must work perfectly, every time, before anyone starts the next phase. If any gate item fails, fix it first. Do not carry broken foundations forward.

Rules of engagement for the whole project:

- **Trunk-based Git workflow:** short-lived feature branches, pull requests reviewed by at least one teammate, main branch always deployable.
- **Environment discipline:** all secrets (API keys, tokens, DB passwords) live in *.env* files that are git-ignored. A committed *.env.example* lists every required variable. Nothing secret is ever hard-coded.
- **Every external API gets a sandbox first:** Meta test app, Pathao/RedX sandbox credentials, SSLCOMMERZ sandbox. Production credentials only after the sandbox flow passes its gate.
- **Tenant safety is non-negotiable:** every table with merchant data carries a *tenant_id*; every query is scoped by it; RLS enforces it at the DB layer. Any feature that breaks tenant isolation is treated as a release blocker.
- **Demo-first mindset:** the judges will see a 5-minute live demo. Anything flaky in that path (login -> inbox -> AI parse -> risk score -> courier booking -> QR payment -> inventory update) has top priority at all times.

Phase Roadmap at a Glance

Phase	Name	Primary Outcome	Duration
0	**Foundations & DevOps Setup**	Repos, environments, CI, database with RLS skeleton running	1 week
1	**Multi-Tenant Core & Auth**	Merchants can register, log in, and see an empty isolated workspace	1–2 weeks
2	**Messaging Ingestion & Unified Inbox**	Real messages from Meta/WhatsApp appear live in the dashboard	2 weeks
3	**AI Order Parser**	One click converts a chat into a validated draft order	1–2 weeks
4	**Inventory & Order Management**	Central stock ledger, atomic decrements, low-stock alerts	1–2 weeks
5	**Courier Dispatch Integration**	Orders booked with Pathao/RedX/Paperfly, tracking synced back	2 weeks

6	**Payments, Bangla QR & Ledger**	QR invoice paid in sandbox settles atomically into the ledger	2 weeks
7	**COD Risk Predictor + Hardening & Demo Prep**	Risk scores on every order; load-tested, demo-rehearsed build	2 weeks

Table 1: Build roadmap. Total: ~12–14 weeks with a 3-person team working in parallel where phases allow.

PHASE 0 Foundations & DevOps Setup

Duration: **1 week**
Depends on: **Nothing**

Goal

Every developer can clone, run, and deploy the full stack locally in under 15 minutes. Nothing product-related exists yet — this phase is purely about eliminating environment friction for the rest of the project.

Tasks

- Create the monorepo structure: */apps/web* (Next.js), */apps/api* (Node.js + Express, TypeScript), */apps/ai* (Python FastAPI), */packages/shared* (types, validation schemas).
- Write *docker-compose.yml* that boots PostgreSQL 16, Redis 7, the API, the AI service, and the web app with one command.
- Set up the migration tool (e.g., node-pg-migrate or Prisma Migrate) and create migration #1: the *tenants* and *users* tables plus the RLS policy template.
- Configure CI (GitHub Actions): on every PR run lint, type-check, unit tests for all three apps; block merge on failure.
- Create *.env.example* covering every service; document local setup in *README.md*.
- Provision a shared staging server (single VPS or free-tier cloud) with automatic deploy from the main branch.

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] A brand-new machine goes from *git clone* to all services running with a single documented command (docker compose up), in under 15 minutes.

[ ] CI pipeline runs on every PR and correctly fails when lint/type/test errors are introduced (verify by pushing a deliberate error).

[ ] Database migrations apply cleanly on an empty database AND re-apply idempotently (up -> down -> up) without manual fixes.

[ ] Main branch auto-deploys to staging; the staging URL serves a health-check endpoint from all three services (web, API, AI).

[ ] No secret value exists anywhere in Git history; .env.example lists every variable each service needs.

PHASE 1 Multi-Tenant Core & Authentication

Duration: **1–2 weeks**
Depends on: **Phase 0**

Goal

The multi-tenant skeleton: merchant sign-up, login, session handling, and bullet-proof tenant isolation. This is the layer every later feature sits on — an isolation bug found later would force a rewrite, so it gets tested hardest now.

Tasks

- Implement merchant registration and login (email + password with bcrypt/argon2 hashing; JWT access token + refresh token, httpOnly cookies).
- Create core tables: *tenants, users, roles* (owner / agent), all business tables carry a non-nullable *tenant_id*.
- Enable PostgreSQL Row-Level Security on every tenant-scoped table; the API sets the tenant context per request (e.g., *SET LOCAL app.tenant_id*) inside a transaction.
- Build API middleware: authentication -> tenant resolution -> role authorization; reject any request without a resolved tenant.
- Frontend shell: login page, protected dashboard layout with sidebar (Inbox, Orders, Inventory, Shipping, Payments, Settings) — all pages empty placeholders.
- Add rate limiting on auth endpoints via Redis; add audit logging of logins.
- Write the tenant-isolation test suite: automated tests that create two tenants and assert every endpoint returns zero rows of the other tenant's data.

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] Two test merchants (Tenant A, Tenant B) can register and log in; sessions survive page refresh; refresh-token rotation works; logout invalidates the session.

[ ] Isolation proof: with seeded data for both tenants, EVERY existing endpoint returns only the caller's rows — verified by the automated cross-tenant test suite AND a manual attempt to fetch Tenant B's record IDs while logged in as Tenant A (must return 404/empty, never 200 with data).

[ ] RLS verified at the database layer directly: a raw SQL session with Tenant A's context cannot read or update Tenant B's rows even when bypassing the API.

[ ] Wrong password, expired token, and missing-token requests all return correct 401/403 responses — never a 500 and never data.

[ ] Auth endpoints are rate-limited (verify a burst of failed logins gets throttled).

[ ] All Phase 1 tests run green in CI.

PHASE 2 Messaging Ingestion & Unified Inbox

Duration: **2 weeks**
Depends on: **Phase 1**

Goal

Real conversations flow from Meta (Messenger, Instagram) and WhatsApp into F-ComFlow and appear live on the merchant dashboard. This is the platform's front door and the first thing judges will see.

Tasks

- Register a Meta developer test app; connect a test Facebook Page, Instagram account, and WhatsApp Business sandbox number.

- Build the webhook gateway: GET verification handshake, HMAC-SHA256 signature validation of every POST, instant 200 OK, raw payload pushed to a Redis queue. Invalid signatures are rejected and logged.
- Build background workers that consume the queue: normalize payloads from the three channels into one internal message model, upsert conversation + customer records (per tenant), persist messages.
- Implement Socket.io: workers broadcast new messages to the correct tenant's dashboard room only.
- Build the Unified Inbox UI: conversation list (channel icon, unread badge, last message), thread view, reply box that sends outbound messages via the Graph/WhatsApp APIs.
- Handle agent collaboration: conversation assignment so two agents don't answer the same customer.
- Add queue observability: dead-letter list for failed payloads, retry with backoff, basic metrics (queue depth, processing lag).

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] A message sent to the test Facebook Page, Instagram account, or WhatsApp number appears in the correct merchant's inbox in under 3 seconds, without a page refresh.

[ ] A reply typed in F-ComFlow arrives back on the customer's Messenger/Instagram/WhatsApp app.

[ ] Webhook signature validation proven: a POST with a tampered or missing signature is rejected (verified with a forged request) and never enters the queue.

[ ] Kill test: stop the worker process, send 50 messages, restart the worker — all 50 messages are processed exactly once, in order, none lost, none duplicated.

[ ] Burst test: 100+ webhook events fired in rapid succession are all acknowledged instantly and fully processed; queue drains to zero.

[ ] Tenant routing proven: messages for Merchant A's page never appear in Merchant B's inbox or socket stream.

[ ] Two agents in the same tenant see the same conversation update simultaneously; assignment prevents double-handling.

PHASE 3 AI Order Parser (Gemini Integration)

Duration: **1–2 weeks**
Depends on: **Phase 2**

Goal

The signature feature: one click turns an unstructured Bengali/English/Banglish conversation into a validated draft order. Accuracy and graceful failure matter more than speed of delivery here.

Tasks

- Build the FastAPI endpoint *POST /api/v1/ai/parse-order*: accepts chat history, calls Gemini 1.5 Flash with a schema-constrained prompt, returns validated JSON (name, 11-digit phone, address, district, product, quantity).
- Enforce output validation with Pydantic: phone must match Bangladeshi mobile format (01XXXXXXXXX); district must map to the official 64-district list (build a normalizer for spelling variants); quantity defaults to 1.
- Handle failure modes explicitly: Gemini timeout, malformed JSON, missing fields -> return a structured error the UI can show, never a crash and never silently wrong data.

- Add per-tenant rate limiting and daily quota on AI calls (Redis counters) to control API cost.
- Build the UI flow: “Extract Order” button in the thread view -> loading state -> editable draft-order form pre-filled with parsed fields, low-confidence/missing fields highlighted -> merchant confirms to create the draft order.
- Create the evaluation harness: a curated set of at least 100 real-style Banglish/Bengali/English chat scripts with hand-labelled expected fields; script computes per-field accuracy on every prompt change.

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] On the 100-script evaluation set, field-level extraction accuracy is >= 90% overall, and phone-number accuracy specifically is >= 95% (wrong phone = failed delivery, so it is the strictest field).

[ ] Average end-to-end parse latency ≤ 1.5 s; the UI shows a proper loading state and never freezes.

[ ] Every parsed phone number in the draft form is a valid 11-digit Bangladeshi number; every district maps to the official list — invalid values are impossible to save.

[ ] Forced-failure tests pass: AI service down, Gemini timeout, and garbage chat input each produce a clear, user-readable error and the merchant can fall back to manual entry.

[ ] The confirmed draft order is stored with the correct tenant_id and linked to the source conversation.

[ ] AI quota enforcement works: calls beyond the per-tenant daily limit are rejected with a clear message, not billed silently.

PHASE 4 Inventory & Order Management

Duration: **1–2 weeks**
Depends on: **Phase 3**

Goal

The transactional heart: products, stock, and the order lifecycle. Everything here must be atomic — the double-selling problem F-ComFlow promises to fix is won or lost in this phase.

Tasks

- Build product catalog CRUD: SKU, name, price, stock_quantity, reorder threshold, image.
- Implement the order state machine: *Draft -> Confirmed -> Dispatched -> Delivered / Returned / Cancelled*, with allowed transitions enforced server-side.
- Make stock decrement atomic: confirmation runs in a single DB transaction using row locking (*SELECT … FOR UPDATE*); stock can never go below zero; failed confirmation rolls back everything.
- Restock logic: cancelled and returned orders return quantities to stock in the same transactional pattern.
- Low-stock alerts: when stock crosses the reorder threshold, queue an email/SMS notification (deduplicated — one alert per crossing, not per order).
- Webhook connectors for Shopify and WooCommerce: outbound stock sync on every change; inbound order webhooks decrement central stock.
- Orders dashboard UI: filterable list, status badges, order detail page with timeline of state changes.

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] Race-condition proof: with a product at stock = 1, two simultaneous confirmations (fired concurrently by a script) result in exactly ONE confirmed order and one clean rejection — run 100 iterations, zero oversells.

[ ] Stock can never go negative under any tested sequence of confirm / cancel / return operations; totals always reconcile (initial stock = current stock + sum of confirmed order quantities).

[ ] Illegal state transitions (e.g., Draft -> Delivered, Cancelled -> Dispatched) are rejected by the API with 4xx errors.

[ ] A stock change in F-ComFlow propagates to the connected Shopify/WooCommerce test store in under 1 second; an order placed on the test store decrements central stock.

[ ] Crossing the reorder threshold fires exactly one low-stock alert; restocking and re-crossing fires again.

[ ] All inventory operations respect tenant isolation (re-run the Phase 1 cross-tenant suite — still green).

PHASE 5 Courier Dispatch Integration

Duration: **2 weeks**
Depends on: **Phase 4**

Goal

A confirmed order becomes a booked shipment without leaving the dashboard: compare carrier rates, book, print the label, and track status automatically.

Tasks

- Obtain sandbox credentials for Pathao, RedX, and Paperfly; implement OAuth/token management with automatic refresh and Redis-cached tokens.
- Design a carrier adapter interface (one internal contract, three implementations) so a fourth carrier is a plug-in, not a rewrite.
- Rate comparison endpoint: given weight + destination district, query all three carriers in parallel and return normalized quotes.
- Booking flow: build the consignment payload from the order, submit, store returned tracking code and carrier, transition order to Dispatched, generate a printable label (PDF).
- Status sync: consume carrier webhooks where offered; poll on a schedule where not; map each carrier's status vocabulary to F-ComFlow's canonical statuses.
- Failure handling: carrier API down or rejecting -> order stays Confirmed with a retriable error surfaced in the UI; no phantom Dispatched states.
- Automated customer SMS on dispatch containing carrier + tracking code (bulk SMS provider integration).

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] End-to-end in sandbox: confirm order -> compare rates from all three carriers -> book with one -> tracking code stored -> label PDF downloads and contains correct recipient data.

[ ] A carrier status change in the sandbox reflects in F-ComFlow within the sync interval, and the order timeline shows it.

[ ] Chaos test: with one carrier's API forced to fail (wrong credentials / blocked endpoint), rate comparison still returns the other two, booking with the dead carrier fails cleanly, and the order remains in a consistent Confirmed state.

[ ] Booking the same order twice is impossible (idempotency guard) — double-clicking Book creates exactly one consignment.

[ ] Token expiry handled: force-expire a cached carrier token and verify the next call transparently refreshes and succeeds.

[ ] Dispatch SMS arrives at a test number with the correct tracking code.

PHASE 6 Payments, Bangla QR & Settlement Ledger

Duration: **2 weeks**
Depends on: **Phase 5**

Goal

Money in, correctly, every time: Bangla QR invoices through SSLCOMMERZ sandbox, webhook-driven confirmation, and an always-consistent settlement ledger. Financial code gets the strictest correctness bar in the project.

Tasks

- Integrate SSLCOMMERZ sandbox: create payment session for an invoice, render the interoperable Bangla QR on the digital invoice page.
- Build the payment webhook endpoint with signature/IPN validation; process confirmations through the atomic settlement transaction: lock order row -> mark Paid -> decrement any remaining reserved stock -> insert ledger entry (gross, 1% MDR + 15% VAT on fee, net).
- Make the webhook idempotent: duplicate IPN deliveries for the same transaction must be detected (unique transaction ID constraint) and processed exactly once.
- Partial advance payments: support collecting a configurable booking fee for high-risk COD orders (foundation for Phase 7).
- Merchant ledger UI: settlement list, running balance, per-order fee breakdown, CSV export.
- Reconciliation job: nightly task compares ledger totals against order records and flags any mismatch.

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] Sandbox happy path: scan/redirect from the QR invoice -> pay with sandbox wallet -> order flips to Paid and the ledger entry appears with mathematically correct gross/fee/net (hand-verify the arithmetic on 5 orders).

[ ] Idempotency proof: replay the same payment webhook 10 times — exactly one ledger entry exists, stock decremented exactly once, no errors leak to the customer.

[ ] Atomicity proof: force a failure mid-transaction (e.g., raise on ledger insert in a test) — order status, stock, and ledger all roll back together; no half-paid states exist.

[ ] A payment webhook for an unknown or already-cancelled order is rejected and logged, never crashes the worker.

[ ] Ledger reconciliation job runs and reports zero discrepancies on seeded test data — and correctly flags a deliberately corrupted row.

[ ] Amounts are stored as exact decimals (never floats); currency rounding verified to 2 decimal places everywhere.

PHASE 7 COD Risk Predictor + Hardening & Demo Prep

Duration: **2 weeks**
Depends on: **Phases 3–6**

Goal

Layer the risk intelligence on top of the finished pipeline, then make the whole system demo-proof: load-tested, monitored, seeded with believable data, and rehearsed against the fest's 5-minute format.

Tasks — Risk Predictor

- Build the feature pipeline: address completeness score, phone validity, customer historical completion rate, regional risk parameters.
- Train the XGBoost classifier on the synthetic 10k-transaction dataset; persist the model artifact with a version tag; serve via the FastAPI service.
- Wire scoring into the order flow: every order gets a risk percentage before dispatch; orders above the configurable threshold show a high-risk banner and a one-click "request advance payment" action (uses Phase 6 partial payments).
- Log every prediction with its feature vector for future retraining on real data.

Tasks — Hardening & Demo Prep

- Load test with JMeter: sustain 120+ webhook events/second for 10 minutes against staging; fix anything that drops payloads or degrades the dashboard.
- Add monitoring: error tracking (e.g., Sentry), uptime checks, queue-depth alerts.
- Full regression pass: re-run every previous phase's exit-gate checklist on the final build.
- Create the demo tenant: realistic seeded products, conversations, and history; script the 5-minute demo path end-to-end; prepare an offline fallback (local stack + recorded video) in case venue internet fails — the rulebook holds teams responsible for demo conditions.
- Rehearse: every team member can run the full demo alone and answer implementation questions on any module (judges may ask anyone).

EXIT GATE — everything below MUST work perfectly before starting the next phase

[ ] Model quality: AUC >= 0.78 on the held-out test split; scores are reproducible from the versioned model artifact.

[ ] Every new order displays a risk score in under 500 ms after confirmation; the high-risk flow (banner -> advance-payment request -> partial payment) works end-to-end in sandbox.

[ ] Risk service down = graceful degradation: orders proceed with a "score unavailable" badge; dispatch is never blocked by a scoring outage.

[ ] Load test passes: 120+ events/sec for 10 minutes, zero lost payloads, dashboard stays responsive, queue drains after the burst.

[ ] Full regression: ALL exit-gate checklists from Phases 0–6 pass again on the release build.

[ ] The 5-minute demo path (login -> live message -> AI parse -> risk score -> courier booking -> QR payment -> inventory + ledger update) runs flawlessly 3 times in a row, including once on the offline fallback stack.

[ ] Every team member has individually rehearsed the demo and the Q&A; on all modules.

Appendix A — Definition of "Works Perfectly"

For every exit-gate item in this guide, "works" means all of the following, not just a successful happy path on the author's laptop:

- **Repeatable:** passes at least 3 consecutive runs, including one from a clean environment (fresh clone or staging).
- **Verified by someone else:** a teammate who did not write the code executes the check and signs it off.
- **Failure-aware:** the documented failure modes (service down, bad input, duplicate event) behave as specified — clear errors, no data corruption, no crashes.
- **Tenant-safe:** the feature has been exercised with two tenants and leaks nothing across them.
- **In CI:** wherever automatable, the check exists as a test that runs on every pull request, so it can never silently regress.

If a later phase breaks an earlier gate, the earlier gate wins: stop feature work and restore it. The exit gates are the project's contract with itself.

End of Developer Implementation Guide